A Vision-Driven Closed-Loop Framework for High-Precision Robotic Baggage Handling

***Abstract*—Robotic baggage handling in airport logistics requires not only reliable grasp execution but also accurate geometric alignment under high-throughput industrial constraints. Existing vision-based manipulation pipelines usually process perception, geometric modeling, and motion control as weakly coupled modules. As a result, their performance can degrade when baggage tags and straps introduce visual interference, when residual mask errors are amplified during 2D to 3D projection, or when conveyor motion causes temporally unstable observations. This paper presents a closed-loop perception–geometric modeling–control framework for dynamic robotic baggage handling. The proposed system improves 2D instance segmentation through multi-scale feature aggregation and attention-guided mask refinement, enhances 3D geometric reliability using pixel-to-3D cross-consistency filtering and temporal oriented bounding-box (OBB) stabilization, and performs predictive grasp execution through recursive state estimation, coarse-to-fine interception planning, and active state regulation. By coupling instance segmentation and spatio-temporal geometric modeling for accurate baggage perception, motion planning for grasp trajectory generation, and hybrid suction–gripper operation for active state regulation under low-confidence conditions, the framework achieves robust dynamic grasping.** (Reviewer 1, Q2; Reviewer 1, Q3; Reviewer 3, Q5) **Experiments on a real robotic platform show that the general grasp success rate remains consistently high, increasing from 93.6% to 96.4%, while the precision success rate improves from 62.1% to 92.7%. These results indicate that the proposed framework primarily improves pose accuracy and placement reliability rather than merely increasing lift success, supporting precision-oriented baggage handling for downstream loading.** (Reviewer 3, Q3; Reviewer 3, Q4)

***Note to Practitioners*—This paper addresses the practical challenges of automating baggage handling in high-speed airport logistics. While industrial robotic arms are highly capable, they struggle to reliably grasp and densely pack baggage from moving conveyor belts. Baggage often features flexible, distracting attachments, such as identification tags and loose straps, which traditional vision systems mistakenly treat as solid grasping points. Furthermore, the conveyor's continuous motion causes tracking errors, leading to failed lifts or inaccurate placements. To solve these issues, we introduce a tightly integrated framework connecting the robot's perception directly to its motion controls. First, tailored image processing accurately outlines the rigid baggage body while explicitly ignoring flexible tags. Second, the system builds a reliable 3D model by filtering depth-sensor noise and smoothing visual jitters. Finally, it predicts the moving baggage's future path, enabling the robotic arm to synchronize its speed and seamlessly intercept the target. If baggage is stacked in a confusing way, the robot actively rearranges items to improve visibility. In real-world tests, this approach improved the precision of placement from 62.1% to 92.7%. This demonstrates that the framework achieves the high-accuracy positioning required to tightly pack baggage into containers.**

***Index Terms*—Robotic Baggage Handling, Industrial Robotics, Predictive Motion Planning, 3D Pose Estimation.**

I. Introduction

ROBOTIC baggage handling is becoming an important component of intelligent airport logistics, where sorting, loading, and transfer operations must be performed reliably within strict time windows. Vision feedback is essential in such systems because it connects object perception with grasp planning and execution. Classical visual servoing establishes closed-loop control by mapping visual feature errors to robot motion commands [1]. However, in practical baggage handling environments, the dominant bottleneck is often not the control law itself, but the reliability and geometric fidelity of the visual observations used by the controller. Baggage items are visually heterogeneous, frequently occluded, and often equipped with deformable tags, straps, wheels, and handles. These factors make the perception-to-action pipeline substantially more difficult than standard pick-and-place manipulation with regular objects.

Conventional robotic grasping pipelines usually detect the target in RGB images and then fuse depth information for 6D pose or 3D bounding-box estimation. Bounding-box-based detection is efficient but lacks the boundary-level accuracy required for precise manipulation, especially when objects are stacked, partially occluded, or densely arranged. Instance segmentation improves spatial localization by separating objects at the pixel level, but segmentation errors at the boundary may still be amplified after projection into the point cloud. In baggage scenarios, flexible tags and straps are particularly problematic: they may be segmented as part of the rigid body in the image, but they do not belong to the stable geometric structure needed for grasping and placement.

Airport baggage handling further imposes several coupled challenges. First, visual interference arises from highly salient and deformable components such as identification tags, ropes, ribbons, and straps. General purpose segmentation models [2] may regard these components either as independent objects or as part of the suitcase body, which shifts the estimated centroid and degrades grasp accuracy. Second, 3D projection distortion occurs when residual 2D mask leakage produces outliers in the projected point cloud. Such outliers can bias PCA-based pose estimation and OBB construction [3], [4]. Third, temporal instability is caused by conveyor motion, sliding, rolling, collision-induced disturbances, motion blur, and depth-sensor noise. These factors introduce frame-to-frame jitter that directly affects predictive grasping. Finally, high-tempo industrial constraints require the robot to grasp moving targets within a short time window. In our system, the conveyor speed reaches approximately $0.5\ \mathrm{m/s}$ (Reviewer 2, Q5). Downstream loading also requires high space utilization, so even small pose errors or volume-estimation errors may reduce packing density and increase the number of required loading cycles.

Suction grasping is attractive for dense baggage scenes because it requires limited lateral insertion space, unlike a mechanical gripper that often needs side clearance. Nevertheless, suction grasping is sensitive to local surface localization and pose accuracy. A successful suction grasp therefore depends on reliable segmentation, stable 3D modeling, and accurate motion prediction [5]. In this work, a dedicated hybrid end-effector is designed that integrates both suction and a gripper mechanism. The gripper is the primary mode for normal dynamic grasping. Suction is used for coarse relocation during active state regulation when confidence in the target grasp is low, adjusting the object pose before state re-estimation and grasp-trajectory replanning. (Reviewer 1, Q3) This hardware configuration motivates a closed-loop software architecture in which instance segmentation, spatio-temporal geometric modeling, motion planning, and active state regulation are coordinated. (Reviewer 1, Q2; Reviewer 3, Q5)

To address the above issues, this paper proposes a vision-driven closed-loop perception, geometric modeling and control framework for high-precision robotic baggage handling. At the perception level, a Multi-Scale Feature Aggregation and Attention-Guided Mask Optimization Network (MSAM-Net) is developed using a YOLOv10 backbone to obtain accurate instance masks under tag and strap interference. (Reviewer 1, Q1; Reviewer 3, Q1) At the geometric modeling level, a pixel-to-3D Cross Consistency Conditional Filtering (PX3C) method removes residual flexible outliers after 2D-to-3D projection within a PCL pipeline with PCA-based OBB estimation, and a Consistency-Aware OBB Stabilization (CAOS) strategy enforces temporal smoothness across frames. (Reviewer 1, Q1; Reviewer 3, Q1) At the action level, a discrete Kalman-filter-based recursive Bayesian estimator predicts target states, a two-stage relative-velocity planning strategy executes rapid interception followed by precise synchronization, and an active state-regulation mechanism resets highly uncertain baggage configurations when passive observation is unreliable.

The contributions of this paper are summarized as follows.

- A real-time semantic perception network, MSAM-Net, is proposed for baggage instance segmentation under severe visual interference. The network combines four-level multi-branch feature aggregation with attention-guided multi-scale mask refinement.
- A spatio-temporal geometric modeling framework is introduced. PX3C removes residual 3D interference by enforcing pixel-to-3D geometric consistency, and CAOS stabilizes OBB pose and scale estimates using quality-aware temporal smoothing.
- A closed-loop dynamic grasping strategy is developed by integrating recursive Bayesian state estimation, two-stage relative-velocity planning, and active state regulation. The complete system is validated on a real robotic baggage-handling platform.

II. Related Work

(Reviewer 1, Q4)

A. Visual Servoing for Robotic Manipulation

Visual servoing is a closed-loop control paradigm in which visual measurements are used to reduce the error between the current and desired robot states [6], and has been widely used in robotics [7], [8]. Depending on the error space, visual servoing is usually categorized into image-based visual servoing (IBVS) and position-based visual servoing (PBVS) . IBVS defines the control error directly in the image plane and can avoid explicit 3D reconstruction in some tasks, but it may suffer from depth ambiguity, image-Jacobian singularity, and complex camera-motion coupling. PBVS [9] defines the error in 3D Cartesian space and therefore supports explicit trajectory planning, which is useful for industrial manipulation. However, PBVS requires reliable 3D pose estimation [6], [10]. In unstructured baggage handling, this requirement becomes the main limitation because visual observations are noisy, dynamic, and affected by flexible attachments. Therefore, a robust perception and geometric modeling frontend is a prerequisite for stable PBVS-style manipulation in this work.

B. Object Detection and Instance Segmentation

Accurate object perception is the foundation of robotic manipulation. Traditional 2D detectors such as Faster R-CNN [11] and 3D detectors such as PointNet++ [12] and PV-RCNN [13] primarily output bounding boxes. Although bounding boxes are computationally efficient, they provide only coarse geometric support and can introduce significant extrapolation errors when baggage items overlap or touch each other. Instance segmentation methods, including Mask R-CNN [2], YOLACT [14], and YOLO-based segmentation models [15], provide pixel-level object separation and are therefore more suitable for downstream point-cloud extraction. Foundation models such as Segment Anything Model (SAM) [16] show strong generalization, but their computational cost is usually too high for embedded real-time robotic control. In this paper, a lightweight YOLOv10-based architecture is modified for baggage segmentation so that real-time inference can be maintained while the specific tag-interference problem is addressed.

C. 3D Geometric Modeling and Point-Cloud Refinement

After obtaining the 2D target mask, robotic grasping requires accurate 3D geometric modeling to estimate the object pose and dimensions. Classical pipelines usually rely on the Point Cloud Library (PCL) [3], which provides standard operations such as VoxelGrid downsampling, statistical outlier removal, and RANSAC-based plane segmentation [17]. The cleaned point cloud is then commonly processed by Principal Component Analysis (PCA) to estimate the principal axes and construct an oriented bounding box (OBB). However, baggage-handling scenarios introduce several adverse sensing conditions. Reflective stickers, luggage tags, flexible straps, and conveyor motion may produce holes, ghost points, and local geometric distortions in depth observations from ToF or structured-light cameras. Therefore, depth completion and point cloud denoising have become important techniques for robust 3D geometric modeling.

Depth completion methods aim to recover missing geometry from incomplete depth measurements. For example, HDCNet [18] adopts a hybrid architecture combining CNN, Transformer, and Mamba modules to infer the geometry of transparent or reflective objects from RGB and depth cues. Such methods can compensate for missing depths caused by specular reflection and reduce hole-induced pose errors. Nevertheless, they typically require large-scale training data and introduce considerable computational overhead. In baggage handling, reflective regions often appear only locally, such as on stickers or tags, while the main suitcase body is usually close to a regular cuboid. Therefore, a lightweight heuristic treatment is more suitable for real-time deployment.

Point cloud denoising has also been widely studied. Traditional filters, such as bilateral filtering [19], [20], can suppress isolated outliers but are less effective for flexible accessories whose geometry does not follow smooth rigid surfaces. Graph-based methods further exploit local geometric similarity, such as feature graph learning with Graph Laplacian regularization [21]. However, these methods still rely heavily on local surface consistency assumptions, which may fail for thin tags and deformable straps. Recent learning-based denoising methods, including Total Denoising [22], [23], and EC-Net [24], improve general denoising performance but are not specifically designed for structured baggage accessories and often require additional computation.

Dynamic point cloud denoising further considers temporal inconsistency caused by object or sensor motion. Xu et al. proposed to use spatio-temporal convolution to aggregate gradient fields across frames [25], while Hu et al. exploited intra-frame self-similarity and inter-frame consistency through spatio-temporal graph learning [26]. These approaches are effective for general dynamic point clouds, but they are usually designed for sensor motion or high-speed targets. In our setting, the camera is fixed and the conveyor speed is controllable. Therefore, instead of employing a heavy dynamic denoising network, this work introduces a task-specific Pixel-to-3D Cross Consistency Conditional Filtering (PX3C) module to remove flexible tags and straps, and a Consistency-Aware OBB Stabilization (CAOS) module to suppress frame-to-frame pose jitter. This design preserves the efficiency of classical PCL-based processing while improving robustness in practical baggage-grasping scenarios.

D. Dynamic Grasping and Active Perception

Grasp planning can be broadly categorized into static and dynamic approaches. Static grasp synthesis assumes that the target object remains stationary during perception and execution, and has been extensively studied in data-driven grasping literature [5]. However, this assumption is not suitable for conveyor-based baggage handling, where objects continuously move, collide, and may undergo pose changes during robot execution. Therefore, robust state estimation and time-aware motion planning are essential for dynamic grasping.

For moving target tracking, recursive Bayesian filtering provides a standard and efficient framework. Learning-based trajectory prediction methods, such as Social LSTM [27], can model complex nonlinear motions, but they usually require large-scale training data and are unnecessary when the target motion is physically constrained. In conveyor scenarios, baggage motion is mainly governed by belt velocity with moderate sensing noise. Thus, a discrete Kalman filter with a constant-acceleration motion model [28] offers a favorable trade-off between robustness and computational efficiency. For dynamic interception, Cowley et al. proposed a perception-and-planning framework for picking moving objects [29]. Inspired by this idea, we adopt a two-stage hierarchical strategy that decouples fast global interception in Cartesian space from local synchronization in relative velocity space, thereby balancing execution speed and grasping precision.

Chen et al. combine global-to-local visual detection with static-to-dynamic planning for moving objects in dense conveyor clutter [30]. Their approach emphasizes adaptive observation and grasp planning, whereas the present framework targets attachment-induced geometric errors and precision placement through OBB refinement and state regulation. (Reviewer 3, Q2)

Another limitation of many existing grasping systems is their reliance on passive perception. When the target enters a chaotic or high-uncertainty state, such as unstable stacking or severe occlusion, passive state estimation may become unreliable. Interactive perception has shown that robot actions can actively reveal occluded geometry and improve perception reliability [31]. Related studies on robotic rearrangement further aim to reorganize cluttered scenes to facilitate downstream manipulation [32]. For example, DexSinGrasp [33] and OPG-Policy [34] use dexterous hands or end-effectors to separate the target from surrounding clutter during grasping. Murray et al. further proposed an interactive visual failure prediction strategy, where probing actions such as lifting or shaking are used before grasp execution [35]. Although these methods mainly focus on static tabletop scenes, they motivate our active intervention strategy for conveyor-based grasping. Instead of simply stopping and retrying after failure, our system actively resets unstable target states and reduces scene uncertainty in real time.

GraspView selects informative viewpoints and aligns multi-view RGB reconstructions for cluttered grasping [36]. Here, the camera remains fixed and active state regulation changes the baggage configuration through suction-assisted relocation, followed by renewed perception and planning. (Reviewer 3, Q2)

Recently, Vision-Language-Action (VLA) models have attracted increasing attention in embodied AI by unifying perception, language understanding, and action generation. Representative examples include RT-2 [37], the Physical Intelligence π series [38], [39], and OpenVLA [40]. These models demonstrate strong potential in semantic reasoning and cross-task generalization. ~~However, they typically involve billions of parameters, require large-scale training data, and often have limited inference frequency, which makes them difficult to deploy in high-speed industrial scenarios with strict real-time constraints.~~ Large model size and inference latency remain deployment concerns. Real-time chunking addresses latency through asynchronous execution of diffusion- or flow-based action policies [41]. This development highlights the

importance of coordinating perception and execution; the present system instead uses explicit geometric state estimation and model-based interception on an embedded processor. (Reviewer 3, Q2) Therefore, for the conveyor-based grasping problem addressed in this work, we prioritize a lightweight model-based planning framework that provides predictable latency, explicit state estimation, and reliable real-time control.

III. System Overview and Problem Formulation

A. System Architecture

The proposed system forms a closed-loop perception, geometric modeling and control pipeline, as illustrated in Fig. 1. The input consists of RGB-D images captured above the conveyor. The instance segmentation module (Reviewer 1, Q2) first segments baggage instances and suppresses tag-like visual interference. The spatio-temporal geometric modeling module (Reviewer 1, Q2) projects the selected mask into a point cloud, removes residual 3D outliers, and estimates a temporally stable OBB. The motion planning module (Reviewer 1, Q2) then predicts the target state, plans a coarse-to-fine interception trajectory, and executes grasping. If the uncertainty of the target state becomes too high, the active state-regulation module triggers a reset behavior.

The central design principle is to avoid irreversible error propagation. A segmentation error can create 3D outliers, a distorted OBB can produce an incorrect grasp pose, and an inaccurate pose can further cause poor placement. Therefore, each stage is designed to evaluate and refine the information passed to the next stage. The framework also closes the loop at the physical level: when passive perception is insufficient, the robot actively changes the state of the baggage to recover observability.

B. Task Definition

Let $\mathcal{I}_t = (I_t^{\text{rgb}}, I_t^d)$ denote the RGB-D observation at time t. The system estimates an object state

$$\boldsymbol{b}_t = \{\boldsymbol{p}_t, \boldsymbol{R}_t, \boldsymbol{s}_t\}, \tag{1}$$

where $\boldsymbol{p}_t$, $\boldsymbol{R}_t$, and $\boldsymbol{s}_t$ denote the centroid, orientation, and size of the target baggage, respectively. Given the current robot end-effector state $\boldsymbol{x}_t^{\text{EE}}$ and the predicted target state $\widehat{\boldsymbol{b}}_{t+\Delta}$, the objective is to generate a grasp trajectory that reaches a feasible grasp pose for the hybrid end-effector (Reviewer 1, Q3) with bounded position and yaw errors. In the experiments, precision success requires a position error below 2 cm (Reviewer 2, Q5) and a yaw error below 5°.

IV. Proposed Method

A. Instance Segmentation With MSAM-Net

(Reviewer 1, Q2)

1) Network Design: To balance segmentation accuracy and computational efficiency, MSAM-Net uses YOLOv10 as the backbone and redesigns the segmentation head for baggage scenes. Baggage instances exhibit large scale variation, partial overlap, and fine boundary interference from tags and straps. MSAM-Net therefore contains two task-driven components: a four-level enhanced multi-branch feature pyramid network (4L-EMB-FPN) and an attention-guided multi-scale mask module (MSM). The architecture is shown in Fig. 2.

The 4L-EMB-FPN extracts four pyramid features, P_2–P_5, from the backbone. Low-level features preserve boundary details, whereas high-level features provide semantic context for large objects. To reduce semantic discrepancy between adjacent pyramid levels, a front-end aggregation fusion (FAF) module first aligns and fuses multi-level features. Given pyramid features P_{n-1}, P_n, P_{n+1}, and P'_{n+1}, FAF pools lower-level features, upsamples higher-level features, and concatenates them with P_n to form an enriched representation P'_n. A refined aggregation fusion (RAF) module then further transforms, pools, upsamples, and concatenates these features to obtain P''_n. This two-stage fusion improves both large-instance semantic consistency and fine-grained boundary sensitivity.

2) Attention-Guided Multi-Scale Mask Module: Although the feature pyramid improves semantic representation, accurate mask reconstruction still requires explicit cross-scale interaction. The MSM uses P_2, P_3, and P_4 for mask prediction. Instead of using only interpolation or pooling, attention-guided downsampling (AG-DS) and attention-guided upsampling (AG-US) are used to exchange information between neighboring scales. In AG-DS, P_3 is projected as the query, while P_2 provides the key and value. In AG-US, P_3 is also used as the query, while P_4 provides the key and value. The attention operation is

$$\text{Attention}(\boldsymbol{Q}_x, \boldsymbol{K}_y, \boldsymbol{V}_y) = \text{Softmax}\left(\frac{\boldsymbol{Q}_x \boldsymbol{K}_y^T}{\sqrt{d}}\right)\boldsymbol{V}_y, \tag{2}$$

where $x \in \{\alpha, \beta\}$ indicates the downsampling or upsampling branch, $y \in \{2, 4\}$ denotes the source pyramid level, and d is the feature dimension. The attention outputs are reshaped and merged with the P_3 feature map.

To improve boundary regularity, an edge-aware mean-square error constraint is introduced using the distance field of the ground-truth boundary:

$$L_{\text{MSE}}(\mathcal{B}_{\text{gt}}, \mathcal{B}_{\text{pre}}) = \frac{1}{|\mathcal{B}_{\text{gt}}|}\sum_{\boldsymbol{p}\in\mathcal{B}_{\text{gt}}} d(\boldsymbol{p}, \mathcal{B}_{\text{pre}})^2, \tag{3}$$

where $\mathcal{B}_{\text{gt}}$ and $\mathcal{B}_{\text{pre}}$ denote the ground-truth and predicted boundary-point sets, respectively, and $d(\boldsymbol{p}, \mathcal{B}_{\text{pre}})$ is the shortest distance from point $\boldsymbol{p}$ to the predicted boundary.

3) Training Objective: The total loss combines standard detection losses with the segmentation constraint:

$$L_{\text{total}} = \lambda_{\text{box}} L_{\text{IOU}} + \lambda_{\text{seg}} L_{\text{seg}} + \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{dfl}} L_{\text{dfl}}. \tag{4}$$

Here, L_{IOU} is the CIoU-based box regression loss, L_{seg} is the cropped mask reconstruction loss with the edge-aware boundary term, L_{cls} is the BCE classification loss on task-aligned soft labels, and L_{dfl} is the distribution focal loss for

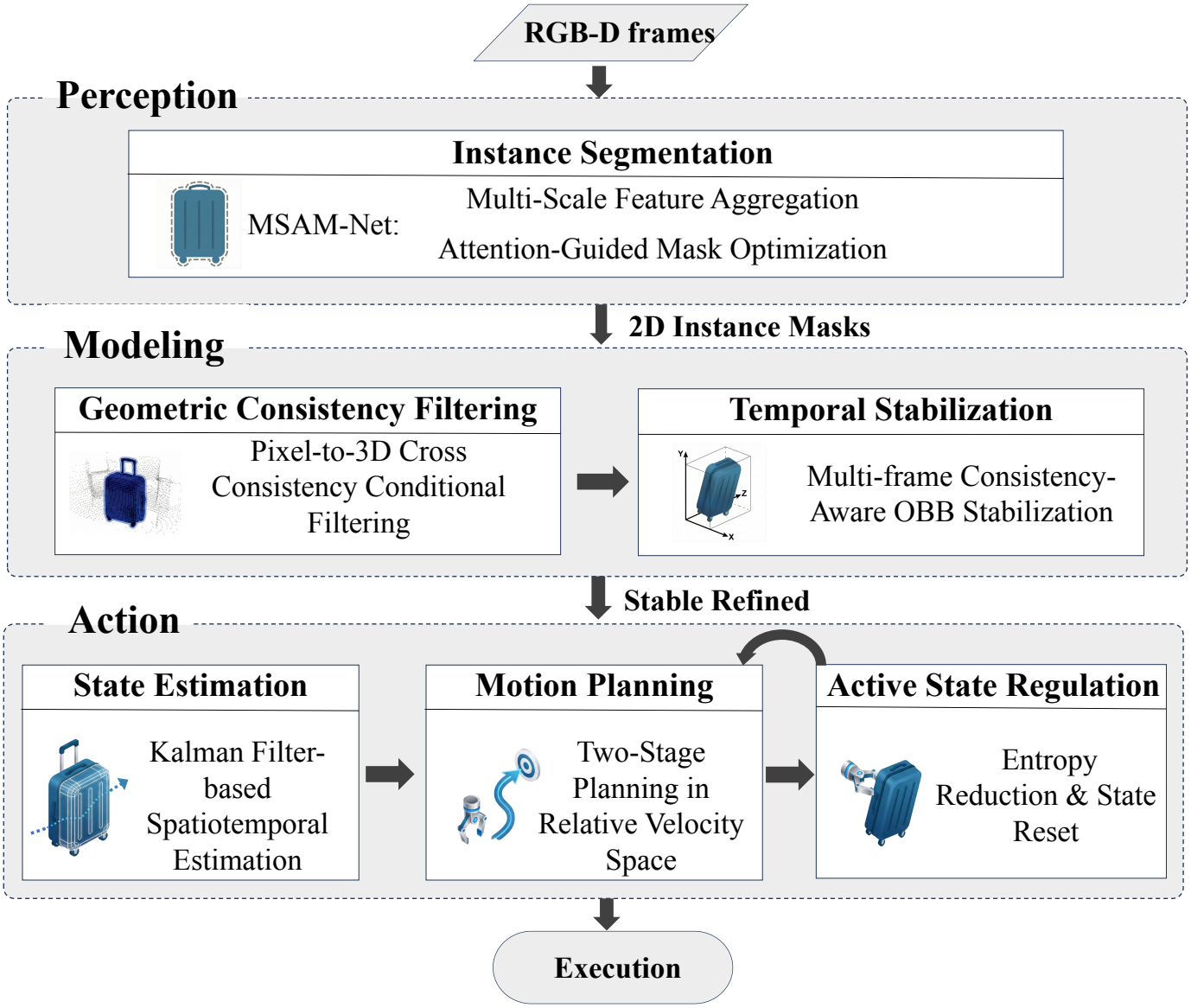


Fig. 1. Overall framework of the proposed closed-loop perception, geometric modeling, and control system. RGB-D frames are first processed through instance segmentation (MSAM-Net) for mask refinement. The resulting point clouds undergo geometric consistency filtering (PX3C) and temporal stabilization (CAOS) to produce a reliable 3D representation. State estimation uses a Kalman-filter-based spatiotemporal approach, followed by two-stage motion planning in relative velocity space. Active state regulation relocates unstable targets to restore observability before re-estimation and grasp planning. (Reviewer 1, Q2; Reviewer 3, Q5)

bounding-box localization. The weights λ_{box}, λ_{seg}, λ_{cls}, and λ_{dfl} balance the four terms.

B. *Spatio-Temporal Geometric Modeling*

(Reviewer 1, Q2)

1) PX3C Filtering: Even after applying the 2D mask from the instance segmentation module (Reviewer 1, Q4), the projected point cloud often contains "ghost" points from unmasked ribbons or tag edges. To resolve this inconsistency between 2D segmentation and 3D geometry, we formulate the filtering process as a Conditional Geometric Self-Consistency problem. We employ standard preprocessing routines from the PCL, utilizing VoxelGrid downsampling for data regularization and RANSAC-based plane segmentation to isolate the rigid baggage body, and apply Principal Component Analysis (PCA) to construct the initial OBB $\mathcal{O}_{\text{cand}}$. However, flexible attachments (Reviewer 1, Q4; Reviewer 2, Q5) can still distort standard PCA, causing the OBB to misalign (e.g., the main axis following a drifting ribbon). This creates a circular dependency: accurate filtering requires a reliable model, but the model itself depends on filtered data. We address this via a self-bootstrapped conditional refinement (Algorithm 1), alternating between provisional model estimation and geometry-aware filtering.

For each point P_i, three features are computed. The thin-slice feature evaluates the local thickness relative to the shortest OBB dimension $L_{\min}$:

$$f_{\text{thin}} = \frac{\max_{p\in\mathcal{N}(i)}\langle \boldsymbol{p}-\bar{\boldsymbol{p}},\boldsymbol{n}\rangle - \min_{p\in\mathcal{N}(i)}\langle \boldsymbol{p}-\bar{\boldsymbol{p}},\boldsymbol{n}\rangle}{L_{\min}}, \quad (5)$$

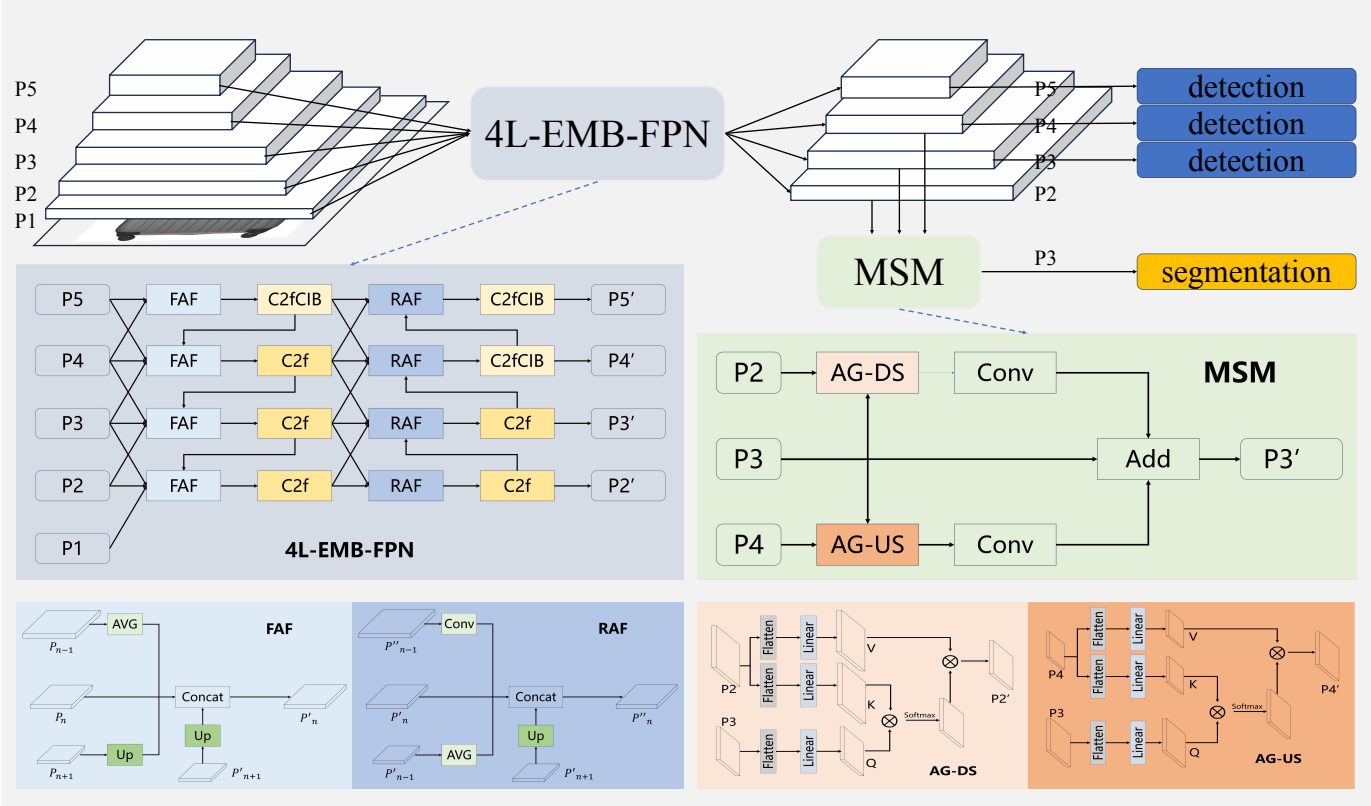


Fig. 2. Schematic overview of the proposed MSAM-Net architecture.

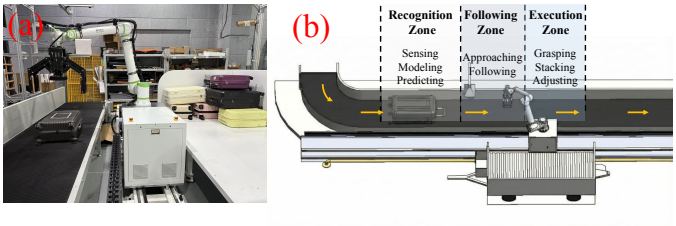


Fig. 3. System Workflow of the Dynamic Grasping Strategy. (Reviewer 1, Q4; Reviewer 2, Q4)

where $\mathcal{N}(i)$ is the local neighborhood, $\bar{\boldsymbol{p}}$ is the neighborhood centroid, and $\boldsymbol{n}$ is the average normal. The protrusion feature measures the normalized distance outside the provisional OBB:

$$f_{\text{prot}} = \max\{0, d_{\text{out}}(i)/\epsilon_{\text{prot}}\}, \tag{6}$$

where $d_{\text{out}}(i)$ is the signed distance to the OBB surface. The normal-consistency feature is

$$f_{\text{norm}} = 1 - |\boldsymbol{n}_i \cdot \bar{\boldsymbol{n}}|, \tag{7}$$

where $\boldsymbol{n}_i$ is the local normal and $\bar{\boldsymbol{n}}$ is the dominant surface normal. Points satisfying the combined flexible-interference condition are removed, and PCA is applied again to reconstruct the refined OBB. This self-bootstrapped refinement reduces the effect of residual 2D segmentation leakage and produces a rigid-body representation more suitable for grasp planning.

2) CAOS Temporal Stabilization: Single-frame OBB estimates may still fluctuate because of motion blur, occlusion, or depth noise. To prevent erratic control commands, we employ the CAOS method (see Algorithm 2), which utilizes multi-frame consistency for stabilization. CAOS stabilizes the OBB sequence using quality-aware temporal smoothing. Let $\mathcal{P}_{\text{filt}}$ be the filtered point cloud and $\mathcal{C}_{\text{main}} \subseteq \mathcal{P}_{\text{filt}}$ be the largest connected component. The coverage score and inlier-rate score are defined as

$$q_{\text{cov}} = \frac{|\mathcal{C}_{\text{main}}|}{|\mathcal{P}_{\text{filt}}|}, \quad q_{\text{in}} = \frac{|\{p \in \mathcal{C}_{\text{main}} : d(p, \mathcal{O}_t) \le \tau_{\text{in}}\}|}{|\mathcal{C}_{\text{main}}|}. \tag{8}$$

The first score measures whether the point cloud is dominated by one coherent object, while the second measures how well the estimated OBB fits the observed points. Temporal stability is quantified as

$$q_{\text{stab}} = \exp\left(-\frac{w_p\|\Delta\boldsymbol{P}\|^2 + w_r\|\Delta\boldsymbol{R}\|^2 + w_s\|\Delta\boldsymbol{S}\|^2}{\eta}\right), \tag{9}$$

where $\Delta\boldsymbol{P}$, $\Delta\boldsymbol{R}$, and $\Delta\boldsymbol{S}$ denote changes in position, rotation, and scale. The aggregate quality score is written as:

$$q = \alpha_1 q_{\text{cov}} + \alpha_2 q_{\text{in}} + \alpha_3 q_{\text{stab}}. \tag{10}$$

and the output OBB state is updated by:

$$\boldsymbol{b}_t = \begin{cases} \boldsymbol{b}^* \oplus (\boldsymbol{v}\Delta t_{\text{gap}}), & q < q_{\min}, \\ (1-\beta(q))\boldsymbol{b}_t^{\text{raw}} + \beta(q)(\boldsymbol{b}_{t-1} \oplus (\boldsymbol{v}\Delta t)), & \text{otherwise}, \end{cases} \tag{11}$$

where $\boldsymbol{b}^*$ is the last reliable snapshot, $\boldsymbol{b}_t^{\text{raw}}$ is the current PX3C observation, $\oplus(\boldsymbol{v}\Delta t)$ denotes conveyor-motion compensation over duration Δt, and $\beta(q) = \min(\beta_{\max}, k(1-q))$ is an adaptive smoothing coefficient. q_{cov}, q_{in}, and q_{stab} denote

Algorithm 1 Pixel-to-3D Cross Consistency Conditional Filtering (PX3C)

Input: Initial point cloud P (masked by 2D results), thresholds $\tau_{\text{thin}}, \tau_{\text{prot}}, \tau_{\text{norm}}$
Output: Refined OBB OBB_{final}, clean point set P'

// Preprocessing Phase
$P_{\text{down}} \leftarrow \text{VoxelGrid}(P)$
$P_{\text{rigid}} \leftarrow \text{RANSAC}(P_{\text{down}})$ ▷ Isolate luggage body
$\text{OBB}_{\text{cand}} \leftarrow \text{PCA}(P_{\text{rigid}})$ ▷ Initial candidate estimation
$L_{\text{min}} \leftarrow \text{ShortestAxis}(\text{OBB}_{\text{cand}})$
$P' \leftarrow \emptyset$
// Conditional Geometric Filtering Loop
for all $p_i \in P_{\text{rigid}}$ **do**
  Compute local neighborhood $\mathcal{N}(i)$, centroid $\bar{p}$, and normal n
  // Feature 1: Thin Slice
  $f_{\text{thin}} = \frac{\max_{p\in\mathcal{N}(i)}\langle \boldsymbol{p}-\bar{\boldsymbol{p}},\boldsymbol{n}\rangle - \min_{p\in\mathcal{N}(i)}\langle \boldsymbol{p}-\bar{\boldsymbol{p}},\boldsymbol{n}\rangle}{L_{\text{min}}}$
  // Feature 2: Protrusion
  $d_{\text{out}}(i) \leftarrow \text{SignedDistance}(p_i, \text{surface}(OBB_{\text{cand}}))$
  $f_{\text{prot}} \leftarrow \max\left\{0, \frac{d_{\text{out}}(i)}{\epsilon_{\text{prot}}}\right\}$
  // Feature 3: Normal Consistency
  $f_{\text{norm}} \leftarrow 1 - |n_i \cdot n_{\text{body}}|$
  // Filtering Condition: Remove if it matches tag/ribbon features
  if $\neg\left(f_{\text{thin}} < \tau_{\text{thin}} \wedge f_{\text{prot}} > \tau_{\text{prot}} \wedge f_{\text{norm}} > \tau_{\text{norm}}\right)$ **then**
    $P' \leftarrow P' \cup \{p_i\}$ ▷ Keep points that are NOT tags/ribbons
  end if
end for
// Refined Reconstruction
$\text{OBB}_{\text{final}} \leftarrow \text{PCA}(P')$
return $\text{OBB}_{\text{final}}$, P'

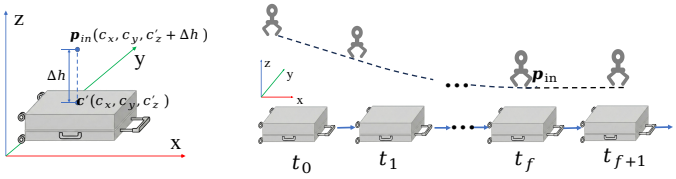


Fig. 4. Schematic Diagram of the Interception Point P_{in} and Rapid Interception Strategy.

coverage, inlier rate, and temporal stability, respectively, and q is their weighted quality score. (Reviewer 1, Q8) A lower quality score increases the reliance on historical prediction, thereby preventing abrupt and unreliable control commands.

C. Dynamic Grasping and Active State Regulation

(Reviewer 1, Q2)

We adopt a "Predict-Follow-Grasp" execution paradigm, as illustrated in Fig. 3. The process involves visual perception and modeling in the Recognition Zone, followed by velocity synchronization in the Following Zone, and finally, grasping execution in the Execution Zone.

Algorithm 2 Consistency-Aware OBB Stabilization (CAOS)

Input: Filtered point cloud P_{filt}, current raw observation b_t^{raw}, previous state b_{t-1}, last stable snapshot b^*, conveyor velocity v, quality threshold q_{min}
Output: Stabilized OBB state b_t

// Quality Assessment Phase
$C_{\text{main}} \leftarrow \text{EuclideanCluster}(P_{\text{filt}})$ ▷ Largest connected component
$q_{\text{cov}} \leftarrow \frac{|C_{\text{main}}|}{|P_{\text{filt}}|}$
$q_{\text{in}} \leftarrow \frac{|\{p\in C_{\text{main}} : d(p,O_t)\le\tau_{\text{in}}\}|}{|C_{\text{main}}|}$

$$q_{\text{stab}} \leftarrow \exp\left(-\frac{w_p\|\Delta P\|^2 + w_r\|\Delta R\|^2 + w_s\|\Delta S\|^2}{\eta}\right)$$

$q \leftarrow \alpha_1 q_{\text{cov}} + \alpha_2 q_{\text{in}} + \alpha_3 q_{\text{stab}}$
// Adaptive Smoothing Weight
$\beta(q) \leftarrow \min(\beta_{\max}, k(1-q))$
// Dynamic State Update Strategy
if $q < q_{\text{min}}$ **then**
  // Low Quality: Extrapolate from Last Stable State
  $\Delta t_{\text{gap}} \leftarrow \text{TimeSince}(b^*)$
  $b_t \leftarrow b^* \oplus (v \cdot \Delta t_{\text{gap}})$
else
  // High Quality: Adaptive Smoothing

$$b_t \leftarrow (1-\beta(q))\, b_t^{\text{raw}} + \beta(q)\,(b_{t-1} \oplus (v\cdot\Delta t))$$

  $b^* \leftarrow b_t$ ▷ Update reference snapshot
end if
return b_t

1) Recursive Bayesian State Estimation: The dynamic state of the baggage centroid is estimated using a discrete Kalman filter with a constant-acceleration model. The state vector is

$$\boldsymbol{x}_k = [p_x, v_x, a_x, p_y, v_y, a_y, p_z, v_z, a_z]^T. \tag{12}$$

The process and observation models are

$$\begin{cases} \boldsymbol{x}_k = \boldsymbol{F}\boldsymbol{x}_{k-1} + \boldsymbol{w}_{k-1}, \\ \boldsymbol{z}_k = \boldsymbol{H}\boldsymbol{x}_k + \boldsymbol{v}_k, \end{cases} \tag{13}$$

where $\boldsymbol{z}_k$ is the observed centroid, $\boldsymbol{w}_{k-1} \sim \mathcal{N}(0, \boldsymbol{Q})$ is the process noise, and $\boldsymbol{v}_k \sim \mathcal{N}(0, \boldsymbol{R})$ is the measurement noise. $\boldsymbol{H}$ is the observation matrix that extracts the position components from the state vector. (Reviewer 1, Q8) The transition matrix is block diagonal:

$$\boldsymbol{F} = \text{diag}(\boldsymbol{F}_x, \boldsymbol{F}_y, \boldsymbol{F}_z), \tag{14}$$

with

$$\boldsymbol{F}_x = \boldsymbol{F}_y = \boldsymbol{F}_z = \begin{bmatrix} 1 & \Delta T & \frac{1}{2}\Delta T^2 \\ 0 & 1 & \Delta T \\ 0 & 0 & 1 \end{bmatrix}. \tag{15}$$

(Reviewer 1, Q8)

The filter operates in a *Predict-Update* cycle. The prior and posterior state estimates are $\hat{\boldsymbol{x}}_k^-$ and $\hat{\boldsymbol{x}}_k$, with error covariances $\boldsymbol{P}_k^-$ and $\boldsymbol{P}_k$, respectively. (Reviewer 1, Q8) In the *Predict* phase, we extrapolate the prior state $\hat{\boldsymbol{x}}_k^-$ and the error covariance $\boldsymbol{P}_k^-$

based on the optimal estimation $\hat{\boldsymbol{x}}_{k-1}$ from the previous step: (Reviewer 1, Q8)

$$\hat{\boldsymbol{x}}_k^- = \boldsymbol{F}\hat{\boldsymbol{x}}_{k-1}, \ \boldsymbol{P}_k^- = \boldsymbol{F}\boldsymbol{P}_{k-1}\boldsymbol{F}^T + \boldsymbol{Q}, \tag{16}$$

and in the *Update* phase, the Kalman Gain $\boldsymbol{K}_k$ is computed using the current observation $\boldsymbol{z}_k$ to correct the prior error, yielding the posterior estimate $\hat{\boldsymbol{x}}_k$ (Reviewer 1, Q8) and the updated posterior error covariance $\boldsymbol{P}_k$ as:

$$\begin{cases} \boldsymbol{K}_k = \boldsymbol{P}_k^- \boldsymbol{H}^T (\boldsymbol{H}\boldsymbol{P}_k^- \boldsymbol{H}^T + \boldsymbol{R})^{-1}, \\ \hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}_k(\boldsymbol{z}_k - \boldsymbol{H}\hat{\boldsymbol{x}}_k^-), \\ \boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}_k\boldsymbol{H})\boldsymbol{P}_k^-. \end{cases} \tag{17}$$

The trace of $\boldsymbol{P}_k$ is used as an uncertainty indicator. When collision, sliding, or occlusion causes the observation model to fail, the uncertainty increases and can trigger active state regulation. This serves as a robust metric to determine whether the target is maintaining stable motion.

For rotation prediction, since rotation on conveyor belts is typically gradual, we apply the same smoothing filter as the position estimation to the components of the quaternion $\boldsymbol{q}_t$. Finally, the target position for a future timestamp t_p is computed as:

$$\boldsymbol{c}(t_p) = \hat{\boldsymbol{p}}_{\text{now}} + \hat{\boldsymbol{v}}_{\text{now}}(t_p - t_{\text{now}}) + \frac{1}{2}\hat{\boldsymbol{a}}_{\text{now}}(t_p - t_{\text{now}})^2. \tag{18}$$

Quaternion components $\boldsymbol{q}(t_p)$ are normalized before conversion to the rotation matrix with $\boldsymbol{R}(t_p) = \text{quat2rot}(\frac{\boldsymbol{q}(t_p)}{\|\boldsymbol{q}(t_p)\|})$ for the robot controller.

2) Coarse-to-Fine Planning in Relative Velocity Space: The grasping planner contains two stages, as shown in Algorithm 3. Stage I performs rapid interception. The end-effector moves from $\boldsymbol{p}_{\text{start}}$ to an "interception point" $\boldsymbol{p}_{\text{in}}$ above the predicted target trajectory in minimal time (as shown in Fig. 4). Rather than solving a full dynamic trajectory optimization problem, the earliest interception time t_f is estimated from

$$\|\boldsymbol{p}(t_f) - \boldsymbol{p}_{\text{start}}\| - \bar{v}(t_f - t_{\text{start}}) = 0, \tag{19}$$

(Reviewer 1, Q8)

where $\bar{v}$ (Reviewer 1, Q8) is the average feasible robot velocity. (19) is solved iteratively using Newton–Raphson updates. To compensate for neglected acceleration and jerk constraints, a buffer term is added:

$$t_f' = t_f + \Delta t_{\text{buffer}}, \quad \Delta t_{\text{buffer}} = \alpha \frac{v_{\max}}{a_{\max}}, \tag{20}$$

where α is a safety factor set to 1.2.

Stage II performs fine synchronization in relative velocity space. The dynamic grasping task is converted into a relative alignment problem by minimizing

$$E_{\text{sync}}(t) = \frac{\|\boldsymbol{v}_{\text{EE}}(t) - \boldsymbol{v}_{\text{grasp}}(t)\|^2}{v_{\text{ref}}^2} + \lambda \frac{\|\boldsymbol{p}_{\text{EE}}^{xy}(t) - \boldsymbol{p}_{\text{grasp}}^{xy}(t)\|^2}{d_{\text{ref}}^2}. \tag{21}$$

The first term measures velocity synchronization, and the second term measures planar position alignment. When $E_{\text{sync}} < \epsilon_{\text{safe}}$, the end-effector and target are treated as relatively static, and the robot performs the final short vertical descent. Because the descent distance is only 80 mm, the residual relative velocity is negligible during this interval. (Reviewer 2, Q5) The final motion is therefore planned as a local point-to-point trajectory using a quintic polynomial.

Algorithm 3 Two-Stage Dynamic Grasping Planning Strategy

Input: Target state estimate posterior state $\hat{\boldsymbol{x}}_k$ (position, velocity, and acceleration); interception settings $\Delta h, \bar{v}$; robot kinematic limits $v_{\max}, a_{\max}$; (Reviewer 1, Q6; Reviewer 1, Q8; Reviewer 2, Q1) thresholds $\epsilon_{\text{safe}}, \lambda, v_{\text{ref}}, d_{\text{ref}}$

Output: Robot position command vector $\mathbf{q}_{\text{cmd}}$

// Stage 1: Fast Interception (Coarse Planning)
Define interception offset point $P_{\text{in}} = \mathbf{p}_{\text{obj}} + [0, 0, \Delta h]^T$
Initialize t_f using heuristic estimation
repeat
  Update t_f using Newton–Raphson on distance constraint:
  $f(t) = \|\mathbf{p}_{\text{obj}}(t) - P_{\text{start}}\| - \bar{v} \cdot t$
  (Reviewer 1, Q8)
until convergence
Apply buffer term: $t_{\text{inter}} \leftarrow t_f + \alpha \frac{v_{\max}}{a_{\max}}$
Execute trajectory to reach P_{in} at time t_{inter}
// Stage 2: Synchronization (Relative Velocity Space)
while grasp not completed **do**
  Obtain real-time error vectors $\mathbf{v}_{\text{err}}, \mathbf{p}_{\text{err}}$
  Compute normalized synchronization cost E_{sync}:
  $E_{\text{sync}} \leftarrow \frac{\|\mathbf{v}_{\text{err}}\|^2}{v_{\text{ref}}^2} + \lambda \frac{\|\mathbf{p}_{\text{err}}\|^2}{d_{\text{ref}}^2}$
  if $E_{\text{sync}} < \epsilon_{\text{safe}}$ **then** ▷ Condition met: relative static state
    Trigger Z-axis descent to target surface
    Close gripper
    return success
  else
    Compute control input to minimize E_{sync}
    Update $\mathbf{q}_{\text{cmd}}$ via inverse kinematics
  end if
end while

3) Active State Regulation: Passive visual servoing assumes that observation uncertainty will eventually decrease as additional frames are received. This assumption is not always valid in conveyor-belt baggage handling. Stacked or colliding objects may remain poorly observable or prevent synchronization. Such configurations can lead to high-entropy object states and increased control uncertainty. (Reviewer 1, Q6; Reviewer 2, Q1; Reviewer 3, Q5) The proposed active state-regulation mechanism (Reviewer 1, Q2) addresses this failure mode by allowing the robot to reshape the environment. When $\text{tr}(\boldsymbol{P}_{p,k}) > \tau_P$ or E_{sync} does not converge, precision placement is temporarily aborted. Here, $\boldsymbol{P}_{p,k}$ is the covariance submatrix for $[p_x, p_y, p_z]^T$. We set $\tau_P = 4.0 \times 10^{-4}$ m^2, corresponding to a root-sum-of-variances positional uncertainty of 0.020 m. (Reviewer 1, Q6; Reviewer 2, Q1) The robot then performs a best-effort suction grasp based on the predicted centroid and relocates the baggage to an empty conveyor region detected by the vision system. This reset action converts a chaotic or poorly observable state into an ordered state for subsequent handling, thereby closing the loop between perception and manipulation. After relocation, the system acquires new RGB-

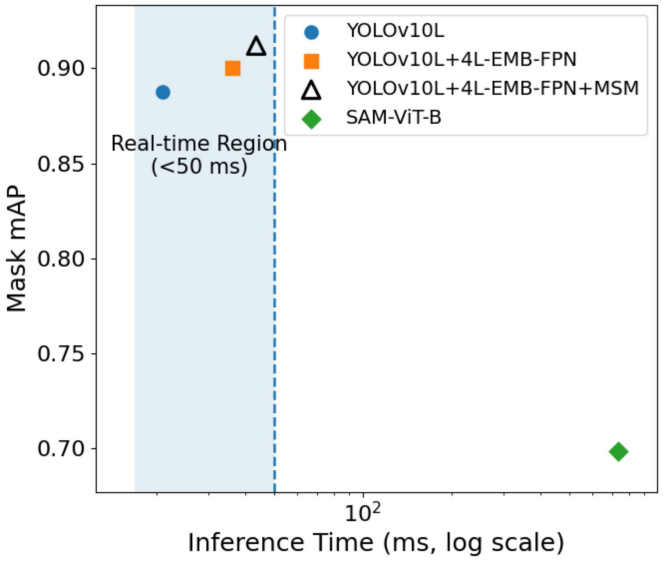


Fig. 5. Mask mAP versus inference time for different segmentation models.

D observations, re-estimates the target state, and replans normal gripper-based grasping. (Reviewer 1, Q3; Reviewer 3, Q5)

V. Experiments and Results

A. Experimental Setup

The proposed framework was deployed on a real robotic baggage-handling platform. The platform consists of a collaborative robotic arm equipped with a hybrid suction–gripper end-effector, a conveyor belt operating at 493.2 mm/s (Reviewer 1, Q6; Reviewer 2, Q1), and an Orbbec Gemini 435Le RGB-D camera with a resolution of 640×400 at 20 FPS. All algorithms were executed on an embedded AX650N processor. MSAM-Net inference, PX3C filtering, and CAOS stabilization were executed at every frame. DKF state updates were executed at 5 Hz ($\Delta T = 0.20$ s), corresponding to a conveyor displacement of approximately 98.6 mm between updates. (Reviewer 1, Q6; Reviewer 2, Q1)

The baggage distribution was designed according to airport statistics collected from three airports across eight flights, including 515 items. Suitcases accounted for 94% of the samples, soft bags for 5%, and irregular objects for 1%. More than 90% of the samples contained strong tag interference. To simulate operational difficulty, random scenarios were generated, including single baggage placement (83%), two-item contact (10%), three-item stacking (2%), and sidewall occlusion (5%).

For 2D segmentation evaluation, SegData comprises SegData-train (49,359 images) and SegData-test (10,344 images), captured under different illumination, viewpoints, and distances. The splits were grouped by baggage instance, with no acquisition sequence shared between training and testing. Initial manual annotations were independently checked by a second annotator, and disagreements were corrected. (Reviewer 1, Q7; Reviewer 2, Q2) For 3D modeling evaluation, OBBData-test was constructed from 1,209 suitcase samples selected from SegData-test. The ground-truth OBBs were manually checked and refined against the corresponding 3D point clouds, followed by independent review and correction of disagreements. (Reviewer 2, Q2) Suitcases were selected because they dominate the baggage distribution and provide stable centroid and yaw references, whereas soft bags are deformable and difficult to annotate consistently.

TABLE I
Core Parameters of Geometric Modeling, Motion Planning, and Active State Regulation. (Reviewer 1, Q6; Reviewer 2, Q1)

Parameter	Value
PX3C	
τ_{thin}	2.0
τ_{prot}	0.2
τ_{norm}	0.065
CAOS	
$\alpha_1, \alpha_2, \alpha_3$	0.43, 0.21, 0.36
$q_{\min}$	0.65
$\beta_{\max}, k$	0.70, 2
DKF, Motion Planning, and Active State Regulation	
$\boldsymbol{Q}$	$\text{diag}([10^{-5}, 10^{-4}, 10^{-3}]_{\times 3})$
$\boldsymbol{R}$	$\text{diag}(2.5\times10^{-5}, 2.5\times10^{-5}, 10^{-4})$ m^2
$\lambda, \epsilon_{\text{safe}}$	0.01, 0.01
τ_P	4.0×10^{-4} m^2

Table I summarizes the core parameters. PX3C used a 0.005 m voxel size, a 0.005 m RANSAC distance threshold with 500 hypotheses per frame, and a 5% candidate-OBB trim fraction. Neighborhood filtering used a 0.03 m radius and at least three neighbors, with $\epsilon_{\text{prot}} = 0.005$ m. CAOS used a 0.15 m clustering tolerance and a minimum cluster size of 500, with $\tau_{\text{in}} = 0.010$ m, $(w_p, w_r, w_s) = (2.0, 1.06, 0.38)$, and $\eta = 1.0$. The voxel, RANSAC, trimming, and conditional-filter settings followed experiment records; neighborhood and clustering settings were fixed implementation choices. (Reviewer 1, Q6; Reviewer 1, Q8; Reviewer 2, Q1)

$\boldsymbol{Q}$ was initialized using the constant-acceleration model and adjusted through preliminary tracking experiments, whereas $\boldsymbol{R}$ was estimated from repeated static RGB-D measurements. Planning used $\Delta h = 0.080$ m, $\bar{v} = 1.0$ m/s, a Newton–Raphson tolerance of 10^{-3} s, and $\alpha = 1.2$ in (20). The interception height followed the required clearance, and synchronization references followed the conveyor speed and positioning tolerance: $v_{\text{ref}} = 0.4932$ m/s and $d_{\text{ref}} = 0.020$ m. The remaining planning and active-regulation settings were selected through preliminary experiments. All settings were kept fixed during evaluation. (Reviewer 1, Q6; Reviewer 2, Q1)

B. Segmentation Performance

Three models were trained on SegData-train and evaluated on SegData-test: YOLOv10L, YOLOv10L with 4L-EMB-FPN, and YOLOv10L with both 4L-EMB-FPN and MSM, i.e., MSAM-Net. The weights were set to $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{seg}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, and $\lambda_{\text{dfl}} = 1.5$, with $L_{\text{seg}} = L_{\text{mask}} + 0.1 L_{\text{MSE}}$, where L_{mask} denotes the cropped mask reconstruction loss and L_{MSE} is defined in (3). These values were selected by grid search, with the loss-weight allocation guided by the official

TABLE II
SEGMENTATION PERFORMANCE ON SEGDATA-TEST.

Method	Box mAP	Mask mAP
YOLOv10L	0.9185	0.8877
+ 4L-EMB-FPN	0.9417	0.9004
+ MSM (MSAM-Net)	0.9524	0.9120

TABLE III
INFERENCE TIME COMPARISON ON AX650N.

Method	Inference time (ms)
YOLOv10L	21.1
+ 4L-EMB-FPN	36.1
+ MSM (MSAM-Net)	43.6
SAM-ViT-B	733.6

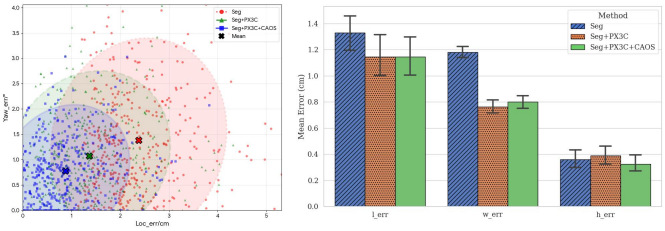


Fig. 6. Comparison of pose and dimension estimation errors before and after PX3C and CAOS. Left: localization and yaw error distributions, with ellipses denoting 95% confidence regions. Right: dimension estimation errors, with error bars indicating 95% confidence intervals.

YOLO implementation. (Reviewer 1, Q6; Reviewer 2, Q1) Table II reports the quantitative segmentation results.

The 4L-EMB-FPN improves mask mAP by 1.27% compared with the YOLOv10L baseline, and the full MSAM-Net further increases the gain to 2.43%. These results show that enhanced multi-scale aggregation and attention-guided mask refinement improve segmentation quality under tag and strap interference.

Inference latency on the AX650N processor is reported in Table III. MSAM-Net increases the inference time from 21.1 ms (Reviewer 2, Q5) to 43.6 ms (Reviewer 2, Q5), but remains below the 50 ms (Reviewer 2, Q5) real-time threshold required for 20 Hz conveyor-based manipulation. By comparison, SAM-ViT-B requires 733.6 ms (Reviewer 2, Q5) per frame and is therefore unsuitable for embedded real-time deployment in this system (as shown in Fig. 5). (Reviewer 1, Q4) SAM was evaluated without task-specific fine-tuning and achieved a mask mAP of 0.69. However, the comparison primarily demonstrates computational feasibility under industrial constraints.

C. 3D Modeling Accuracy

To evaluate PX3C and CAOS, dimension and pose estimation errors were computed on OBBData-test. Dimension error was defined as the absolute error in length, width, and height. Pose error included centroid localization error and yaw error in the X–Y plane. Fig. 6 compares pose and dimension estimation before and after PX3C and CAOS.

Compared with segmentation-only modeling, PX3C reduces the width estimation error by 35.5%. The combined PX3C+CAOS pipeline further stabilizes the estimates. The length estimation error decreases by approximately 13.8%, and the height estimation error reaches its minimum value of 0.33 cm (Reviewer 2, Q5) when both modules are applied. The overall downward shift of the confidence intervals indicates consistent improvement.

The pose-error distributions are shown in Fig. 6. Each scatter point corresponds to one sample, and the ellipses denote 95% confidence regions. Localization error decreases from 2.38 cm (Reviewer 2, Q5) using segmentation-only modeling to 1.36 cm (Reviewer 2, Q5) with PX3C and to 0.88 cm (Reviewer 2, Q5) with CAOS. Yaw error decreases from 1.38° to 0.78°. These results confirm that PX3C suppresses geometric outliers and CAOS reduces temporal fluctuations.

Representative qualitative OBB comparisons are shown in Fig. 7. Red boxes denote OBBs derived directly from segmentation, while green boxes denote refined OBBs after PX3C filtering. Flexible tags and ribbons are removed effectively, while rigid components such as handles and wheels are preserved. Temporal stabilization results are shown in Fig. 8, where CAOS reduces abrupt OBB fluctuations across consecutive frames caused by lighting variation, occlusion, and motion blur.

D. System-Level Grasping Performance

Two metrics were used to evaluate system-level performance. General Success Rate (GSR) is the number of successful target grasps without dropping divided by all trials. Precision Success Rate (PSR) is the proportion of precise suitcase placements among successful suitcase grasps. We define precise placement as final placement with a position error below 2 cm and a yaw error below 5°, evaluated after the baggage has stabilized. (Reviewer 1, Q5; Reviewer 3, Q4) In multi-object scenarios, the highest object along the Z-axis was selected as the grasping target.

Each of the five configurations was evaluated in 500 trials, for 2500 trials in total. Every configuration included 470 suitcase, 25 backpack, and 5 irregular-object trials, with the same scenario distribution: 83% single-item placement, 10% two-item contact, 2% three-item stacking, and 5% sidewall occlusion. (Reviewer 2, Q3) The snapshots are shown in Fig. 9. An ablation study was performed by incrementally integrating MSAM-Net, PX3C+CAOS, DKF+two-stage planning, and active state regulation. The results are summarized in Table IV. For the full system, the short adjustment window on the moving conveyor limited each trial to at most one active reset. Relocation, renewed perception, replanning, and the subsequent gripper-based grasp belonged to the same trial and were counted once. A failed reset, failed subsequent grasp, or target leaving the robot workspace before grasp completion was counted as a failed trial. Intermediate reset placement was excluded from the final precision assessment. (Reviewer 1, Q5; Reviewer 2, Q3)

The GSR increases moderately from 93.6% to 96.4%, indicating that even the baseline system can lift many objects. In contrast, PSR increases substantially from 62.1% to 92.7%. The gains of 2.8 and 30.6 percentage points, respectively, show

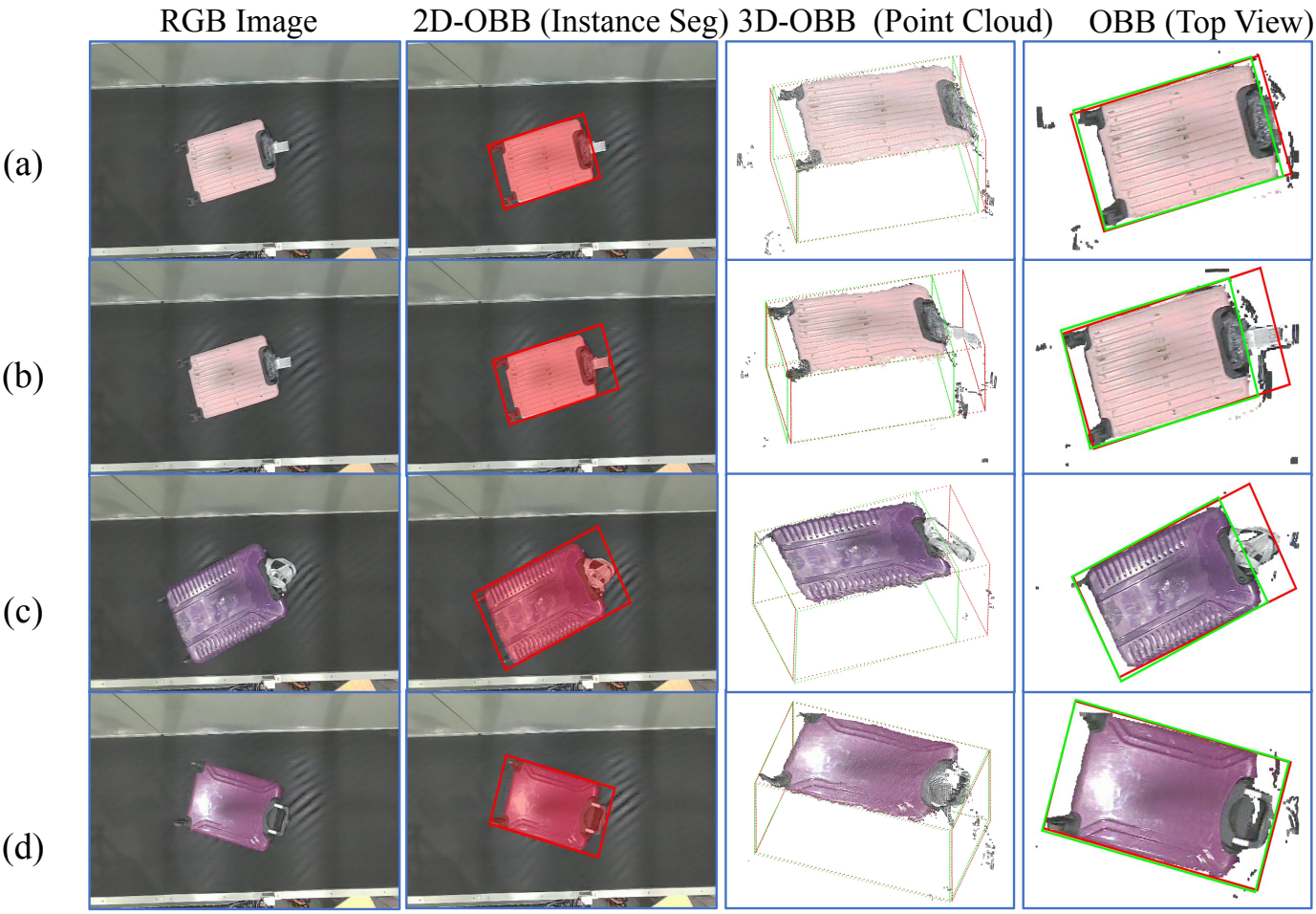


Fig. 7. Qualitative OBB comparison before and after PX3C filtering.

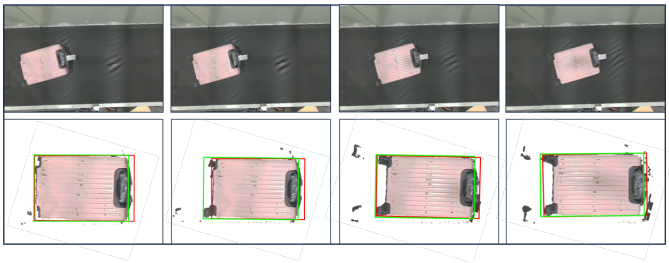


Fig. 8. OBB predictions across consecutive frames with CAOS stabilization.

TABLE IV
ABLATION STUDY WITH 500 TRIALS PER CONFIGURATION.
(Reviewer 1, Q5; Reviewer 2, Q3)

Method	GSR (%)	PSR (%)
Baseline (YOLOv10L + PCL)	93.6	62.1
+ Instance segmentation (MSAM-Net) (Reviewer 1, Q2)	95.2	69.6
+ Geometric modeling (PX3C + CAOS) (Reviewer 1, Q2)	95.6	83.8
+ Predictive control (DKF + two-stage planning) (Reviewer 1, Q2)	96.4	84.9
+ Active State Regulation (full system)	96.4	92.7

that the framework primarily improves placement precision while maintaining a high grasp success rate. (Reviewer 3, Q4)

In practical airport operations, baggage arrives with varying poses and stacking densities, making robust instance segmentation and pose estimation essential for reliable dynamic grasping. Successful lifting alone does not ensure that baggage can be placed according to a planned container layout. The PSR improvement indicates more accurate placement, which can help maintain the intended positions, orientations, and clearances between items. This provides a basis for tighter packing arrangements and higher packing density in subsequent aircraft baggage-loading deployment. (Reviewer 3, Q4)

VI. Conclusion and Future Work

This paper presented a closed-loop framework for dynamic robotic baggage handling in airport logistics environments. The proposed system tightly integrates instance segmentation, spatio-temporal geometric modeling, motion planning, and active state regulation (Reviewer 1, Q2). MSAM-Net improves segmentation robustness under severe tag interference through multi-scale aggregation and attention-guided mask refinement. PX3C and CAOS enhance 3D geometric reliability by suppressing residual outliers and stabilizing temporal OBB estimation. The DKF-based motion planning module and relative-velocity planning strategy (Reviewer 1, Q2) enable stable grasp

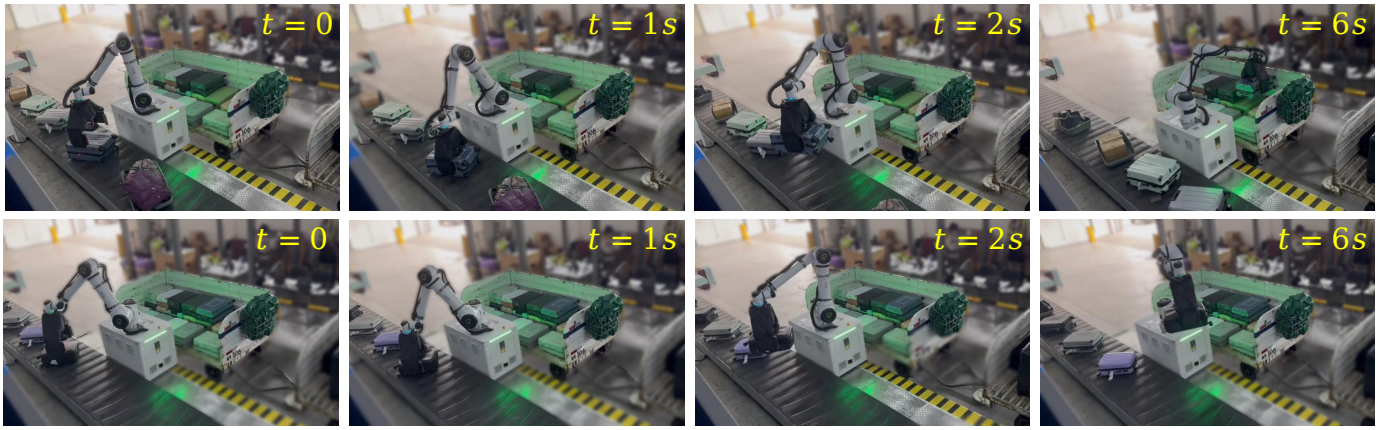


Fig. 9. Snapshots of the dynamic grasping experiment. The proposed framework achieves robust dynamic grasping on a real conveyor belt in an industrial environment, overcoming the challenges of high-speed motion, cluttered scenes, and control uncertainty.

execution under conveyor motion, while active state regulation handles unstable or high-uncertainty configurations.

Experiments on a real robotic platform demonstrate that the proposed framework significantly improves precision-oriented grasping. While the general grasp success rate remains consistently high, the precision success rate among successfully grasped suitcases improves from 62.1% to 92.7%. (Reviewer 1, Q5; Reviewer 3, Q4) This improvement highlights the value of reliable geometric modeling and motion planning for precise placement in downstream baggage loading. (Reviewer 3, Q4) Although this work does not explicitly optimize packing policies, it provides the geometric and execution reliability required by high-density loading systems.

The current validation is limited to predominantly rigid baggage on a structured conveyor at a nearly fixed speed. Performance with deformable baggage, severe clutter, variable conveyor motion, and different airport layouts remains to be established. (Reviewer 3, Q3)

Future work will integrate tactile sensing to improve contact feedback and the handling of deformable baggage. We will also investigate packing planning that coordinates container layouts with grasping and placement to improve packing density in practical aircraft baggage loading. (Reviewer 3, Q3; Reviewer 3, Q4)

REFERENCES

[1] S. Hutchinson, G. D. Hager *et al.*, "A tutorial on visual servo control," *IEEE Trans. Robot. Autom.*, vol. 12, no. 5, pp. 651–670, 2002.

[2] K. He, G. Gkioxari *et al.*, "Mask r-cnn," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2961–2969.

[3] R. B. Rusu and S. Cousins, "3d is here: Point cloud library (pcl)," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 1–4.

[4] S. Gottschalk, M. C. Lin *et al.*, "Obbtree: A hierarchical structure for rapid interference detection," in *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*, 1996, pp. 171–180.

[5] J. Bohg, A. Morales *et al.*, "Data-driven grasp synthesis—a survey," *IEEE Trans. Robot.*, vol. 30, no. 2, pp. 289–309, 2013.

[6] F. Chaumette and S. Hutchinson, "Visual servoing and visual tracking," *Handb. Robot.*, pp. 563–583, 2008.

[7] R. Wang, Z. Zhu *et al.*, "Visual servoing trajectory tracking and depth identification for mobile robots with velocity saturation constraints," *IEEE Trans. Ind. Electron.*, vol. 71, no. 6, pp. 5950–5959, 2023.

[8] G. He, Y. Jangir *et al.*, "Image-based visual servo control for aerial manipulation using a fully-actuated uav," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 5042–5049.

[9] V. Lippiello, B. Siciliano *et al.*, "Position-based visual servoing in industrial multirobot cells using a hybrid camera configuration," *IEEE Trans. Robot.*, vol. 23, no. 1, pp. 73–86, 2007.

[10] N. Zhu, W.-F. Xie *et al.*, "Position-based visual servoing of a 6-rss parallel robot using adaptive sliding mode control," *ISA Trans.*, vol. 144, pp. 398–408, 2024.

[11] S. Ren, K. He *et al.*, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[12] C. R. Qi, L. Yi *et al.*, "Pointnet++: Deep hierarchical feature learning on point sets in a metric space," *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[13] S. Shi, C. Guo *et al.*, "Pv-rcnn: Point-voxel feature set abstraction for 3d object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 10 529–10 538.

[14] D. Bolya, C. Zhou *et al.*, "Yolact: Real-time instance segmentation," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9157–9166.

[15] G. Jocher, J. Qiu *et al.*, "Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models," 2026. [Online]. Available: https://arxiv.org/abs/2606.03748

[16] A. Kirillov, E. Mintun *et al.*, "Segment anything," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 4015–4026.

[17] M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[18] G. Xie, M. Li *et al.*, "Hdcnet: A hybrid depth completion network for grasping transparent and reflective objects," *arXiv preprint arXiv:2511.07081*, 2025.

[19] J. Digne and C. De Franchis, "The bilateral filter for point clouds," *Image Process. On Line*, vol. 7, pp. 278–287, 2017.

[20] H. Si, Z. Wei *et al.*, "Lbf: learnable bilateral filter for point cloud denoising," *arXiv preprint arXiv:2210.15950*, 2022.

[21] W. Hu, X. Gao *et al.*, "Feature graph learning for 3d point cloud denoising," *IEEE Trans. Signal Process.*, vol. 68, pp. 2841–2856, 2020.

[22] P. Hermosilla, T. Ritschel *et al.*, "Total denoising: Unsupervised learning of 3d point cloud cleaning," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 52–60.

[23] M.-J. Rakotosaona, V. La Barbera *et al.*, "Pointcleannet: Learning to denoise and remove outliers from dense point clouds," in *Comput. Graph. Forum*, vol. 39, no. 1. Wiley Online Library, 2020, pp. 185–203.

[24] L. Yu, X. Li *et al.*, "Ec-net: an edge-aware point set consolidation network," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 386–402.

[25] Q. Hu and W. Hu, "Dynamic point cloud denoising via gradient fields," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 21, no. 4, pp. 1–24, 2025.

[26] W. Hu, Q. Hu *et al.*, "Dynamic point cloud denoising via manifold-to-manifold distance," *IEEE Trans. Image Process.*, vol. 30, pp. 6168–6183, 2021.
[27] A. Alahi, K. Goel *et al.*, "Social lstm: Human trajectory prediction in crowded spaces," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 961–971.
[28] X. R. Li and V. P. Jilkov, "Survey of maneuvering target tracking. part i. dynamic models," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 4, pp. 1333–1364, 2003.
[29] A. Cowley, B. Cohen *et al.*, "Perception and motion planning for pick-and-place of dynamic objects," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 816–823.
[30] (Reviewer 3, Q2) H. Chen, T. Kiyokawa *et al.*, "Adaptive grasping of moving objects in dense clutter via global-to-local detection and static-to-dynamic planning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
[31] J. Bohg, K. Hausman *et al.*, "Interactive perception: Leveraging action in perception and perception in action," *IEEE Trans. Robot.*, vol. 33, no. 6, pp. 1273–1291, 2017.
[32] D. Batra, A. X. Chang *et al.*, "Rearrangement: A challenge for embodied ai," *arXiv preprint arXiv:2011.01975*, 2020.
[33] L. Xu, Z. Liu *et al.*, "Dexsingrasp: Learning a unified policy for dexterous object singulation and grasping in densely cluttered environments," *IEEE Robot. Autom. Lett.*, vol. 11, no. 2, pp. 1346–1353, 2025.
[34] H. Ding, Y. Zeng *et al.*, "Opg-policy: Occluded push-grasp policy learning with amodal segmentation," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 7257–7263.
[35] M. Murray, A. Gupta *et al.*, "Learning to grasp in clutter with interactive visual failure prediction," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 18 172–18 178.
[36] (Reviewer 3, Q2) S. Wang, M. Dai *et al.*, "GraspView: Active perception scoring and best-view optimization for robotic grasping in cluttered environments," *arXiv preprint arXiv:2511.04199*, 2025.
[37] B. Zitkovich, T. Yu *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
[38] K. Black, N. Brown *et al.*, "π_0: A Vision-Language-Action Flow Model for General Robot Control," *arXiv preprint arXiv:2410.24164*, 2024.
[39] Physical Intelligence, K. Black *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.
[40] M. J. Kim, K. Pertsch *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
[41] (Reviewer 3, Q2) K. Black, M. Y. Galliker *et al.*, "Real-time execution of action chunking flow policies," in *Adv. Neural Inf. Process. Syst.*, 2025. [Online]. Available: https://arxiv.org/abs/2506.07339